

13_resume_star

Section 10: Resume Bullets

Every bullet below describes the same project — Helios, my Autonomous Growth Diagnosis Engine on the Google Merchandise Store GA4 public dataset. The numbers are consistent across surfaces (85% vs 45% root-cause accuracy, <5 min/run, 0 hallucinated columns, 50 labeled scenarios, 5 MCP servers, 7 agents, a 28-metric / 16-dimension semantic layer) so an interviewer who reads my resume, my LinkedIn, and my one-pager hears one coherent story.

5 resume bullets

These are tuned for ATS parsing: action-verb-led, one quantified outcome each, canonical keywords spelled out.

- **Architected** an autonomous **7-agent** AI Growth Analyst (Claude Agent SDK, Model Context Protocol) that diagnoses *why* an e-commerce funnel moved, achieving **85% root-cause accuracy vs 45%** for a naive largest-delta baseline on a **50-scenario** offline labeled benchmark.
- **Engineered** a mix-shift-vs-rate-change decomposition (mix / rate / interaction) over GA4 funnel metrics that resolves Simpson's paradox, computed deterministically and validated to **<=10% MAPE** against analytically-derived ground truth.
- **Built** a governed-SQL grounding layer across **5 MCP servers** with a **28-metric, 16-dimension** semantic registry as the sole SQL author, eliminating LLM hallucination to **0 hallucinated columns** while holding query cost under a fixed **5 GiB** BigQuery byte budget.
- **Designed** the dbt analytics layer (staging -> intermediate -> marts) on the GA4 BigQuery export with session-grain reconciliation tests (**<=0.5% drift**) and a **GitHub Actions CI** eval gate blocking any accuracy regression.
- **Delivered** time-to-diagnosis **<5 min/run**, with **100%** of findings carrying a significance test, a dollar revenue-at-risk figure, and a prioritized, power-analyzed experiment — each adversarially vetted by a Critic agent before release.

10 LinkedIn bullets

Slightly more narrative and first-person — these read like someone talking about work they're proud of, not a keyword block.

- I built Helios, an always-on "AI Growth Analyst" that runs on a schedule and proactively tells you *why* an e-commerce funnel moved — not another dashboard, and not a SQL chatbot.
- The technical heart is a mix-shift-vs-rate-change decomposition: I split every aggregate rate change into a traffic-composition effect and an in-segment-behavior effect, which is exactly what dissolves Simpson's paradox in real funnel data.
- I made the system *trustworthy*, not just clever: the LLM never writes raw SQL and never computes a statistic — it composes governed metrics through a semantic layer and calls deterministic Python (scipy / statsmodels / prophet) over 5 MCP servers.
- To prove it works I designed an offline labeled benchmark — 50 scenarios across 7 buckets — by injecting *known* anomalies into a frozen GA4 copy and grading whether the pipeline rediscovers the segment I hid.
- The headline result: 85% root-cause-segment accuracy versus 45% for the obvious "largest absolute delta" baseline, with that gap regression-gated in CI on every pull request.
- I orchestrated 7 agents on a deterministic finite-state machine — Orchestrator, Monitor, Decompose, Diagnose, Prescribe, Narrator, and an adversarial Critic that tries to refute every finding before it ships.
- I used the right model for each job: Opus for the hard reasoning roles (orchestration, diagnosis, critique) and Sonnet for the high-volume worker steps, to balance quality against token cost.
- Every finding Helios emits is *actionable* by construction — it carries a significance test, dollars of revenue-at-risk, and a powered experiment card, never a naked assertion.
- I modeled the GA4 export end to end in dbt and handled the classic gotchas honestly: sessions keyed on `(user_pseudo_id, ga_session_id)`, and the `traffic_source` struct being user first-touch rather than session source.
- I was deliberate about limits: the dataset is ~3 months and observational, so Helios *designs and sizes* experiments and runs quasi-experimental readbacks (pre/post, difference-in-differences) rather than pretending to run live A/Bs.

Internship-discussion bullets

These connect my GA4 / GTM / funnel-tracking internship work directly to Helios — useful when an interviewer asks "so what did you actually do at your internship, and how does this project build on it?"

- In my internship I instrumented GA4 / GTM event tracking and built funnel reports; the recurring pain was that the dashboards showed *what* changed but the "why" still took an analyst a day or more — Helios is my answer to that exact gap.
- I'd lived the Simpson's-paradox trap manually — a funnel rate dropping while every segment held flat because mobile/paid traffic mix shifted — so in Helios I encoded that

decomposition as a first-class, deterministic tool instead of a thing analysts re-discover by hand.

- I knew the GA4 BigQuery export's real edges from instrumenting it (event-scoped params, the `traffic_source` first-touch gotcha, no session row to query), so I modeled sessionization and channel grouping correctly from day one rather than learning it the hard way.
- The ecommerce funnel I tracked in GTM (session_start -> view_item -> add_to_cart -> begin_checkout -> purchase) is the same funnel Helios diagnoses — I turned a reporting workflow I knew intimately into an autonomous diagnosis engine.
- Measurement-discipline habits from the internship — reconciling event counts to source totals, distrusting attribution defaults — became Helios's verify-then-trust principles: dry-run cost gating, $\leq 0.5\%$ reconciliation, and an adversarial Critic.

Placement-season bullets

One-liners for a campus CV / one-pager, mapped to the specific role a given company is hiring for. Pick the variant that matches the JD.

- **Product Analyst:** Built an autonomous funnel-diagnosis engine on GA4 that separates traffic-mix shifts from behavior changes and quantifies revenue-at-risk in dollars (85% vs 45% root-cause accuracy on 50 labeled scenarios).
- **Growth Analyst:** Turned conversion-rate drops into segment-attributed root causes and a prioritized, power-analyzed experiment backlog, shipped as an executive Decision Brief in under 5 minutes per run.
- **Data Analyst:** Modeled the GA4 BigQuery export in dbt (staging -> marts) with a governed 28-metric / 16-dimension semantic layer and reconciliation tests holding drift $\leq 0.5\%$.
- **Analytics Engineer:** Designed the dbt DAG and a MetricFlow-aligned semantic registry as the single source of truth for all SQL, with a CI gate enforcing 0 hallucinated columns and a fixed 5 GiB byte budget.
- **AI / Data Engineer:** Orchestrated 7 agents over 5 MCP servers (Claude Agent SDK) with governed SQL and deterministic stats as enforceable trust boundaries, benchmarked at 85% accuracy with an adversarial Critic.

Section 11: STAR Stories

Seven behavioral stories, each in explicit Situation / Task / Action / Result form with a one-line signal of what it demonstrates. I keep them concrete — real decisions, named tradeoffs, honest limits.

Story 1 — Scoping a flagship from a vague brief (project ambiguity)

Situation. I wanted one flagship portfolio project for placement season, and the only fixed inputs were a public dataset (the GA4 obfuscated ecommerce sample) and my own GA4/funnel background. The brief was essentially "build something impressive with analytics + AI" — wide open, and easy to drift into a generic dashboard or yet another text-to-SQL chatbot.

Task. Convert that ambiguity into a sharp, defensible product thesis I could actually ship and explain in an interview.

Action. I started from the pain, not the tech: dashboards report *what* happened; the expensive, judgment-heavy work is *why*, and it takes an analyst 1-3 days per anomaly. I wrote an explicit anti-product stance — "not a dashboard, not a SQL chatbot, autonomous-first" — to keep myself honest, then scoped Helios down to a single load-bearing centerpiece (mix-shift vs rate-change decomposition) and a five-phase maturity ladder (L1 intern MVP through L3 top-1% and two production-frontier phases) so each phase was independently demoable with green CI exit criteria.

Result. A crisp 60-second pitch and a buildable plan: an autonomous engine that diagnoses funnel movement, prices it in dollars, and prescribes experiments. The anti-product stance kept scope from sprawling, and the phase ladder meant I always had a shippable, demoable increment.

Signal: Turns an open-ended brief into a sharp thesis and a phased plan; defines what *not* to build.

Story 2 — The mix-vs-rate decomposition (a hard technical challenge)

Situation. Overall `session_conversion_rate` fell week-over-week. The obvious read — and the one a naive "largest absolute segment delta" heuristic produces — was "checkout is broken, go fix it."

Task. Find the *real* cause before prescribing a fix, and do it in a way I could defend mathematically rather than by intuition.

Action. I implemented `stats-mcp.decompose_change`, which treats an aggregate rate as $R = \sum_i (w_i * r_i)$ and splits `deltaR` into a mix effect $\sum_i (dw_i * r_i(t_0))$, a rate effect $\sum_i (w_i(t_0) * dr_i)$, and an interaction term $\sum_i (dw_i * dr_i)$. I enforced the Simpson's-paradox-safe discipline of computing rates as $\text{SUM}(\text{numerator})/\text{SUM}(\text{denominator})$ after grouping — never averaging per-segment ratios — and unit-tested the function against hand-worked golden values so the algebra was provably correct.

Result. The per-segment rates were essentially flat; the drop was almost entirely *mix* — a surge of low-converting mobile / Paid Social traffic had shifted the weights. The prescription flipped from "fix checkout" to "fix acquisition mix," and the answer was fully auditable. That decomposition became the technical heart of the whole project.

Signal: Goes past the obvious answer to the mathematically correct one; recognizes and resolves Simpson's paradox.

Story 3 — Governed SQL via MCP, deterministic stats (an architecture decision)

Situation. I wanted an LLM in the loop for reasoning, but LLMs do two things that are fatal in an analytics tool: they hallucinate column and metric names, and they "compute" statistics in token-space that look right and aren't.

Task. Eliminate both failure modes *structurally* — not by prompting the model to be careful, but by making the bad outcomes impossible.

Action. I split the system's capabilities across 5 MCP servers and drew hard boundaries: `semantic-mcp` (a governed 28-metric / 16-dimension registry) is the *only* path that emits SQL, so the model composes metric names like `view_to_cart_rate` and never authors raw SQL; `stats-mcp` is the *only* path to math, so every decomposition, significance test, and forecast is real seeded Python. I considered the simpler alternative — let the model write SQL and validate it afterward — and rejected it: validation catches errors after the fact, whereas a registry makes a hallucinated column a hard error at composition time. `warehouse-mcp` holds the sole BigQuery client behind a mandatory dry-run byte gate.

Result. 0 hallucinated columns by construction, every number reproducible, and the architecture itself *is* the trust argument — capability isolation became enforceable trust boundaries rather than hopeful prompting.

Signal: Designs for correctness structurally; names the rejected alternative and why it's weaker.

Story 4 — The labeled benchmark (evaluation design)

Situation. "The AI says so" is not shippable, and the live GA4 data's real anomalies are *unlabeled* — I don't know their true root causes, so I can't grade against them.

Task. Make the system's trustworthiness *measurable* with a number I could put on a resume and defend.

Action. I built an offline labeled benchmark by injecting *known* anomalies into a frozen copy of the data. A seeded Python injector applies one of two surgical primitives — a rate perturbation

(change `r_i` only, holding segment volume `w_i` fixed) or a volume/mix perturbation (change `w_i` only, holding rates fixed) — and writes a ground-truth label, including the analytically-exact mix/rate/interaction split and true dollar-at-risk. I built 50 scenarios across 7 buckets (single/multi rate, single/multi mix, seasonality decoys, no-anomaly controls, data-quality artifacts) and — crucially — I implemented the *naive baseline* too (largest absolute segment delta) so my headline number had an honest comparison point. The whole harness runs in GitHub Actions and gates every PR.

Result. 85% root-cause-segment accuracy vs 45% for the naive baseline, decomposition MAPE $\leq 10\%$, 0 hallucinations — with regressions caught automatically in CI. The no-anomaly controls and seasonality decoys also let me measure false positives, not just hits.

Signal: Knows that a metric without a baseline is meaningless; builds the adversary and the controls, not just the happy path.

Story 5 — Learning MCP, the Agent SDK, and dbt fast (learning new tech)

Situation. Helios required three technologies I hadn't shipped before: the Model Context Protocol, the Claude Agent SDK for multi-agent orchestration, and dbt for the warehouse layer. My prior experience was GA4/GTM analytics, not agent systems or analytics engineering at this depth.

Task. Get productive in all three quickly enough to build a coherent system, without producing a fragile pile of half-understood glue.

Action. I learned each by anchoring it to a concept I already understood. MCP I treated as a typed RPC boundary — which let me reframe it as the *trust boundary* of the whole design rather than just plumbing. The Agent SDK I constrained deliberately: I ran the 7 agents on a deterministic finite-state machine where the LLM only ever *composes tool calls* and never controls flow, which made the system debuggable instead of an emergent mess. dbt I learned through its conventions — staging/intermediate/marts layering, tests-as-first-class — and front-loaded the two keystones (sessionization and the semantic registry) because they fail *silently* if wrong.

Result. A working end-to-end system across all three, and — more importantly — opinions about each: why a constrained FSM beats free-form agent autonomy here, and why the MCP boundary is the feature, not the framework. I learned them well enough to defend the design choices, not just operate the tools.

Signal: Learns unfamiliar tech by mapping it to known concepts; ends up with defensible opinions, not just working code.

Story 6 — Catching my own attribution mistake (handling feedback / self-correction)

Situation. Early on, my channel attribution looked subtly wrong, and separately my interview-prep docs had drifted — different sections cited different scenario counts and slightly different metric framings as the spec evolved.

Task. Find and fix the root cause of the attribution error, and reconcile the documentation so the project told one consistent story.

Action. On attribution, I discovered I had trusted the event-level `traffic_source` struct as the session's source — but in GA4 it's the user's *first-touch* attribution, not session source. I rebuilt `channel_group` on session-scoped `event_params` source/medium with first-touch only as a documented fallback, and I centralized the logic in a single `channel_group` macro so it could never diverge again. On the docs, I treated a Canonical Reference Card as the single source of truth and reconciled every cross-section inconsistency back to it (scenario counts, metric names, the 10 channel groups) rather than patching sections independently.

Result. Correct GA4-style channel grouping, and a documentation set that no longer contradicts itself — one canonical vocabulary, enforced. The fix also became one of my strongest "honest gotcha" talking points in interviews.

Signal: Owns and roots-out their own error; fixes the *class* of bug (single source of truth) rather than the instance.

Story 7 — Deferring LTV under data and budget constraints (prioritization under constraints)

Situation. Two hard constraints bounded what Helios could honestly claim: the dataset is only ~3 months (Nov 2020 - Jan 2021) and observational, and I held myself to a fixed 5 GiB BigQuery byte budget per run with a <5 min/run latency target.

Task. Decide what to build now, what to defer, and — most importantly — how to be honest about it rather than overclaiming.

Action. I explicitly deferred true customer LTV and long-horizon retention: a 3-month window cannot support lifetime curves, so I report short-horizon proxy retention with stated assumptions instead of inventing a number. Because there's no experiment assignment in the data, I scoped Helios to *design and size* experiments and run quasi-experimental readbacks (pre/post, difference-in-differences) rather than claiming live A/B testing. For cost, I made `dry_run` byte-estimation mandatory before every `run_query`, pruned date shards, and pre-aggregated the daily funnel fact so runs stayed under budget. I wrote these deferrals into the roadmap with explicit rationale.

Result. A system that's both performant (<5 min/run, under the 5 GiB budget) and *credible* — every limitation is stated with its reason. In interviews, naming what I deliberately didn't do, and why, reads as senior judgment rather than a gap.

Signal: Prioritizes ruthlessly under real constraints and states limitations as deliberate, reasoned tradeoffs — maturity, not weakness.